Case Study: Video Jockey Feature Development

Real-World Agentic AI Development Over 4 Sessions

Project: StreamSurv Windows Video Surveillance System
Feature: Video Jockey — timestamp seeking and playback speed control for recorded streams
Duration: January 6 - January 29, 2026 (4 active sessions, 1,832 user messages)
Hands-on time: ~20 hours (across ~81 hrs wall clock)
Branch: `ak/feature-jockey-win`

Overview: What Was Built

The Video Jockey feature enables users to:

- **Seek to any timestamp** in recorded video streams without restarting
- **Control playback speed** (0.1x slow-motion to 32x fast-forward)
- **Non-destructive updates** — stream continues without interruption

Architecture Diagram

```
+-----------------------------------------------------------------------------+
|                         VIDEO JOCKEY ARCHITECTURE                           |
+-----------------------------------------------------------------------------+
|                                                                             |
|  +-------------------+    +-------------------+    +--------------------+   |
|  |  Video Player     |<-->|  VideoService     |<-->| Redis Pub/Sub      |   |
|  |  (Browser/App)    |    |  (Cloud API)      |    |                    |   |
|  +-------------------+    +-------------------+    +---------+----------+   |
|                                                              |              |
|                                                              v              |
|  +-------------------------------------------------------------------------+|
|  |                        SITE MANAGER (C#)                                |
|  |  +-----------------------+    +--------------------------------+       |
|  |  |  CloudProxyRedis      |--->|  UpdateRecStreamHandler        |       |
|  |  |  - Subscribe to Redis |    |  - Parse seek commands         |       |
|  |  |  - Route to handlers  |    |  - Send JSON via stdin to C++  |       |
|  |  +-----------------------+    +--------------------------------+       |
|  +-------------------------------------------------------------------------+
|                                          |                                  |
|                                          v                                  |
|  +-------------------------------------------------------------------------+
|  |                      PIPELINE INSTANCE (C++)                            |
|  |  +----------------+   +--------------+   +-------------------------+   |
|  |  | MilestonePull  |-->| H264Frame    |-->| RTSPPusher              |   |
|  |  | ModuleRec      |   | Demuxer      |   | (FFMPEG output)         |   |
|  |  |                |   |              |   |                         |   |
|  |  | - Fetch frames |   | - P-frame    |   | - PTS calculation       |   |
|  |  | - Interruptible|   |   dropping   |   | - Queue management      |   |
```

```
  |  |  |   pacer        |   | - NAL routing|   | - RTSP streaming        |  |   |
  |  |  +----------------+   +--------------+   +-------------------------+  |   |
  |  +-------------------------------------------------------------------------+   |
  |                                                                                |
  +--------------------------------------------------------------------------------+
```

Session Timeline: The Burst Pattern

Session	Date	Active / Wall Clock	Bursts	Focus	Key Outcomes
1	Jan 6	43 min / 43 min	1	**Planning & Architecture**	Created branch, designed IPC protocol, 12 tasks, zero code
2	Jan 7	~5 hrs / 12 hrs	3	**Full C++ Implementation**	Phase 1 complete, VS2022 retargeting, PowerShell test harness
3	Jan 27	~7 hrs / 22 hrs	3	**Debugging Marathon**	PTS overflow, flip-flop bug, playrate, 12 context overflows
4	Jan 27–29	~8 hrs / 47 hrs	10	**Refinement, Testing & CI/CD**	9-sec delay fix, process hang, frame dropping, CI/CD pipeline

The pattern: Work happened in focused bursts (30 min – 3 hrs) fit around evenings, early mornings, and weekends. Between bursts, the system was idle — agent done, waiting for the human. The human is the bottleneck, not the agent.

Pattern 1: Planning as Product (Session 1)

The Opening Move

The developer opened a workflow document and let the agent explore:

> 👤 USER: [opened video-jockey-workflow.md]
>
> 🤖 AGENT: I'll read the file first to understand the jockey feature, then handle the git operations...
>
> [Agent reads docs, explores codebase structure, identifies key files]
>
> I've read the jockey feature documentation. Now let me analyze the codebase to understand what changes are needed. I'll explore the key components mentioned in the workflow...

Key Pattern: Deep Exploration Before Coding

The agent spent significant time understanding the architecture before proposing any changes:

1. Read the workflow document
2. Explored SiteManager C# code
3. Examined PipelineInstance C++ code

4. Identified the critical architecture difference (Windows vs Linux)
5. Created a detailed implementation plan

Human Guidance: Simple Over Clever

When the agent proposed a complex mutex-based approach:

> 👤 USER: I'd suggest using boost as it is already integrated... this is simple command, we should keep it very simple on line commands to go from C# to C++. Fire and forget is OK. No—first create a detailed task plan and define how will we unit test it... I'll sign off before you code.

Workshop Takeaway: The developer insisted on a plan before implementation, established testing requirements upfront, and kept architecture simple.

Pattern 2: Guardrails in Action (Session 4)

The "Slow Down" Moment

After hours of implementation, the agent started jumping to conclusions:

> 👤 USER: hold your horses and analyze the issue clearly... you are jumping like a kid
>
> 🤖 AGENT: You're right. Let me slow down and analyze properly.
>
> **What we know from the stacks:**
>
> | Thread | Location | What it means |
> |---|---|---|
> | Main (177800) | Wait_for_event::wait() | Waiting for modules to signal |
> | Worker (169188) | interruptible_wait | Could be our code or not |
>
> ...

Workshop Takeaway: The developer caught the agent getting hasty and forced structured analysis. The agent responded with a table-based breakdown.

Human as Oracle: The Frame Dropping Insight

When the agent was deep in PTS (Presentation Timestamp) calculations:

> 👤 USER: Let me make sure you understand the purpose of this implementation: at 32x we want the user to be able to quickly coarse review of the video so there is no need for the user to see every frame in quick succession, hence the decision to drop frame, also this has a potential to clog the network bandwidth (32x × 2mbps is a big number, imagine it with 20 other cameras, it gets worse)
>
> Timebase is rescaled to squish the timeline, let's argue if that may be causing some issues on the player side (do we have a way of proving it?) Maybe a better technique is to keep the timebase constant...

Workshop Takeaway: The developer provided business context (bandwidth, user experience) that the agent couldn't infer from code alone.

Pattern 3: Context Management Under Stress

The Dreaded Context Overflow

Session 4 hit context limits **five times**, each time requiring a continuation:

> 👤 USER: This session is being continued from a previous conversation that ran out of context. The summary below covers the earlier portion of the conversation.
>
> Analysis:
>
> 1. **Initial Context (from previous session summary)**:
> - Working on Video Jockey feature
> - Previous work identified two issues: PTS bug and 8.5 second delay
> - Was analyzing if FrameQueue's condition variables could be reused... [truncated]

What Got Lost (and How to Prevent It)

Lost Context	Symptom	Prevention
Architecture decisions	Agent re-explored already-decided patterns	Document in workflow.md
Debugging progress	Re-analyzed same logs	Update progress.md after each finding
Test results	Re-ran tests unnecessarily	Log results in markdown

Recovery Pattern: File-Based State

The project used markdown files as persistent memory:

```
docs/
├── video-jockey-workflow.md    # Architecture decisions
├── jockey-implementation-progress.md  # Task tracking (later removed)
└── CLAUDE.md                   # Project instructions
```

Workshop Takeaway: When context overflows, the agent needs explicit instructions to read state files. Files survive; conversation doesn't.

Pattern 4: Testing with Human Intervention

The 15-Second Pause Investigation

The agent proposed automated tests, but the developer intervened:

> 👤 USER: this is a qualitative test so I'll run it manually, and ask you to analyze logs
>
> 🤖 AGENT: Sounds good. Run the test manually with the sequence:

> 1. Start at 1x (or 32x)
> 2. Switch to 0.1x - let it run for a few seconds
> 3. Switch to 32x - **this is the key moment** - should now be near-instant instead of 8.5s delay
>
> **What to look for in logs:** ...

Log Analysis as Collaboration

> 👤 USER: I ran the test and log is collected. My observation:
>
> 1. we again reached the end before test completed but did not exit normally
> 2. ctrl+c worked
> 3. I saw huge misbehavior switching from 0.1x to 32x—a lot of frames played very soon...
>
> question: 0.1x approach sends I frames and P frames alike, but 32x sends only few I frames. so what happens when there are frames in the pusher queue and we switch from 0.1x to 32x?
>
> 🤖 AGENT: [Analyzes logs, identifies queue buffering issue]
>
> **Log Analysis Summary**
>
> | Switch | From | To | Command Time | Processing Time | Delay |
> |---|---|---|---|---|---|
> | 1 | 32x | 0.1x | 03:42:25.848 | 03:42:25.869 | **21ms** ☑ |
> | 2 | 0.1x | 32x | 03:43:08.568 | 03:43:08.569 | **~1ms** ☑ |
>
> ...

Workshop Takeaway: Human runs tests, agent analyzes results. Division of labor matches human judgment (is this acceptable?) vs agent capability (pattern recognition in logs).

Pattern 5: Learning Extraction → Documentation

From Debug Sessions to Timeless Docs

After completing implementation, the developer insisted on cleanup:

> 👤 USER: update the docs, remove any unwanted details about completed tasks... maintain timeless documentation which can be used by next generation of developers or coding agents to understand the implementation for maintenance and enhancements

Before vs After

Before (Progress Tracking):

```
## Task 2.1: C# Handler Implementation - DONE ✓ (2026-01-28)
- Added UpdateRecStreamHandler to CloudProxyRedis.cs
- Build succeeded at 15:42 UTC
```

```
## Task 2.2: Stdin Writer - PENDING
...
```

After (Reference Documentation):

```
## Seek Command Flow

1. VideoService sends `Update_Rec_Stream_LVSM` via Redis
2. CloudProxyRedis routes to UpdateRecStreamHandler
3. Handler converts timestamps and sends JSON via stdin:
   ```json
   {"cmd":"seek","from":"2024-01-15T10:30:00","rate":2.0}
   ```
4. PipelineInstance parses and applies to MilestonePullModuleRec
```

Workshop Takeaway: Progress tracking is for the session; architecture docs are forever. Convert findings to timeless reference after completion.

Pattern 6: Subagent Usage (Visible in Session Structure)

Evidence of Subagents

The JSONL archive shows subagent folders:

```
927a8e46.../subagents/agent-a701157.jsonl  (116KB)
abd1652f.../subagents/agent-a1df58c.jsonl  (198KB)
abd1652f.../subagents/agent-a3e0f0a.jsonl  (259KB)
```

What Subagents Were Used For

Based on tool-results folders and subagent transcripts, subagents handled:

- **Build verification** — Compile C++ and C# components
- **Log file analysis** — Parse large pipeline logs
- **Test execution** — Run PowerShell test scripts
- **Code audits** — Comprehensive searches across codebase

Example Subagent Task: Timestamp Format Audit

The main agent delegated a comprehensive audit to a subagent:

> SUBAGENT TASK: I need to audit all timestamp format usage in the Video Jockey feature. Please search and analyze:
>
> 1. **C++ side (PipelineInstance.exe):**
> - main.cpp: How does stdinReaderThread parse the JSON seek command?

> - PipeLineInstance.cpp: How does seek() pass timestamps?
> - MilestonePullModuleRec.cpp: What format does it expect?
>
> 2. **C# side (SiteManager):**
>
> - CloudProxyRedis: How are timestamps formatted? ...

This 60-message subagent session searched the entire codebase, identified inconsistencies, and reported findings back to the main agent.

Workshop Takeaway: The main agent stays focused on design/coordination while subagents handle execution tasks. Audit/research tasks are ideal for subagent delegation.

Pattern 7: When the Agent Struggles

The P-Frame Mystery

The agent spent significant effort debugging why P-frames weren't being dropped at 8x playrate:

> 🤖 AGENT: The `mDropPFrames` is a regular `bool`, not atomic. But that's unlikely to cause this issue on most platforms. Let me check where the H264FrameDemuxer is actually created...
>
> [Multiple iterations of analysis]
>
> 🤖 AGENT: I see something interesting. Looking at `try_push()` in FrameQueueAdapter, when `should_push()` returns `DONT_PUSH`, it returns `true` (line 74)—meaning "handled". So the `try_push rc: 1` in the log could mean either "pushed" OR "dropped intentionally"...

The Resolution

After extensive debugging (and adding diagnostic logging):

> 🤖 AGENT: **The frame dropping IS working correctly now!**
>
> | Playrate | mDropPFrames | P-frames Dropped | P-frames Passed |
> |---|---|---|---|
> | 8x | 1 | 20,734 | 0 |
> | 32x | 1 | 20,734 | 0 |

Workshop Takeaway: The agent can get lost in complex debugging. Adding explicit logging (which the human approved) was the breakthrough.

Anti-Patterns Observed

1. Premature Code Changes

> 👤 USER: why are you changing existing code, this has been working for years without any issues. I have still not given go ahead for changes... you should be planning

> 🤖 AGENT: You're right, I apologize. I should not have modified TimeUtils.cs—that was out of scope. Let me revert that change.

Lesson: Agent got ahead of approval. Explicit "plan first, implement after approval" guardrails needed.

2. Losing Track of the Big Picture

> 👤 USER: I think a simple approach => seek command clears the queue is better than anything else. we are already clearing the queue when the start time changes. let's make that code clean and consistent
>
> 🤖 AGENT: Done. Summary of changes:
>
> 1. RTSPPusher.cpp - Always clear queue on seek

Lesson: Human saw the forest; agent was examining bark. Periodic "zoom out" prompts help.

3. Inconsistent Testing

Test script had issues with Ctrl+C handling that took multiple iterations:

> Exchange 22: "Ctrl+C worked for jockey-playrate but not others" Exchange 45: "The stop command was never received by C++" Exchange 59: "Ctrl+C handling has been completely rewritten"

Lesson: Test infrastructure deserves the same rigor as production code.

Metrics: What We Can Measure

Session Statistics

Metric	Value
Total sessions	4 active + 1 snapshot
Total messages	1,832 user + 3,921 assistant = 5,753
Context overflows	25
Wall clock span	~81 hrs
Actual hands-on time	**~20 hrs**
Lines of C++ changed	~800
Lines of C# changed	~400
Files modified	25+

Time Investment (from timestamp analysis)

Activity	Hands-on Time
Planning & architecture	43 min

Activity	Hands-on Time
Implementation (C++ & C#)	~8 hrs
Debugging & testing	~9 hrs
Documentation + CI/CD	~2 hrs
Total	**~20 hours**

Comparison: Without Agentic AI

Akhil's estimate for this feature solo (the task is extremely complex — cross-process IPC, video protocol, C++ and C#):

~3 weeks of focused time (~120 hours)

But critically, in the traditional approach these deliverables would **not exist**:

- **Architecture documentation** — IPC protocol spec, JSON schema, component diagrams
- **Automated test harness** — PowerShell script driving end-to-end playback testing
- **Tribal knowledge docs** — 3 undocumented requirements, now captured in code comments & docs

Estimated multiplier: 6× — and the AI method raised the quality bar.

Zero F5 Debugging

Not a single line of step-through debugging across the entire project. No breakpoints, no watch windows, no F5 hell. Instead: the agent iterated on code and log analysis while Akhil focused on verifying the feature worked *visually* and *reliably* across multitudes of test runs. Higher confidence, zero boredom.

Key Takeaways for Workshop

1. Planning Discipline Pays Off

Session 1 spent 43 minutes on planning with zero code written. This investment paid off in every subsequent session.

2. The Burst Pattern Works

Work happened in focused bursts fit around life — evenings, early mornings, weekends. No marathon sessions. Define tasks, review output, course-correct, walk away. The agent finishes in minutes; you're the bottleneck.

3. Human Judgment Remains Critical

The developer corrected the agent on:

- Architecture simplicity (boost over custom IPC)
- Business context (bandwidth constraints)
- Testing approach (visual verification over step-debugging)
- Code change approval (don't modify without sign-off)

4. Context Management is Real Work

25 context overflows across 4 active sessions. File-based state (workflow.md, CLAUDE.md) was essential for continuity.

5. Debugging is Collaborative, Not Step-Through

Human runs tests and verifies visually. Agent analyzes logs, proposes hypotheses, iterates on code. Zero F5 debugging across the entire project — higher confidence from running multitudes of end-to-end tests.

6. The AI Method Raises the Quality Bar

Architecture docs, automated test harness, tribal knowledge documentation — none of these would have been produced in the traditional solo approach. The 6× multiplier understates the true value.

Appendix: Session Exchange Highlights

Exchange 6: Root Cause Discovery

> 👤 USER: well have you analyzed if the issue is due to the sleep(...) in MilestonePullModule... when we are in the .1x mode, we have a large sleep, if the seek command arrives during this sleep... we have an issue.
>
> 🤖 AGENT: **FOUND IT!** Line 14044: Sleeping for(ms) : 10015 ← 10 SECOND SLEEP! This is at 22:27:30.49. The seek command arrived at 22:27:31.97, but the source thread was **in the middle of a 10-second sleep** and couldn't wake up to check the seek queue!

Exchange 33: The "Slow Down" Moment

> 👤 USER: hold your horses and analyze the issue clearly... you are jumping like a kid

Exchange 53: Simple Solution Over Complex

> 👤 USER: I think a simple approach => seek command clears the queue is better than anything else. we are already clearing the queue when the start time changes. let's make that code clean and consistent

Exchange 97: Documentation as Final Product

> 👤 USER: update the docs, remove any unwanted details about completed tasks... maintain timeless documentation which can be used by next generation of developers or coding agents

---

Case study extracted from Claude Code JSONL transcripts
January-February 2026